COS 214: Practical 6

Members:

Corné Myburgh	**-u25505204**
Dirk Kunz	**-u24689999**
Markus Zeelie	**-u25098544**
Motlatso Makgamatho	**-u25605021**
Hafani Ramagoma	**-u23605091**
Takunda Mugwagwa	**-u24667341**
Amelton khoza	**-u19201062**

Task 1: Research Workflow Management Systems

Architecture of the entire system
Hollingsworth, D. (1995). Workflow Management Coalition: The Workflow Reference Model (Technical Report TC00 - 1003). Workflow Management Coalition (WfMC).

Reason:
The source above clarifies the difference between a workflow definition and an instance. It provides the industry standard definition separating a Process Definition from a Process Enactment.
It also outlines the core architecture of a generic workflow engine, defining standard interfaces for client applications, external invoked applications, and administration tools.

Control Flow and Routing Mechanics:
van der Aalst, W.M.P., ter Hofstede, A.H.M., Kiepuszewski, B., & Barros, A.P. (2003). Workflow Patterns. Distributed and Parallel Databases, 12(1), 5, - 45

Reason:
The source above explains how complex activities are modeled as structured execution threads.
It also documents the universal control-flow partners required in generic engines which directly inform how TIVIDY handles process branching activity dependencies.

Participants and Role Allocation Framework
Russell, N., van der Aalst, W. M. P., ter Hofstede, A. H. M., & Edmond, D. (2005). *Workflow Resource Patterns: Identification and Taxonomy of Resource Assignment Approaches*. In *Advanced Information Systems Engineering (CAiSE)*, Springer, 284–299.

Reason:
The sources above explain how work items are allocated to human or system actors.

It also provides clear concepts for modelling participant roles, delegation, escalation rules, and authority hierarchies in manual approval processes.

System Lifecycle and Integration Foundation
Weske, M. (2024). *Business Process Management: Concepts, Languages, Architectures* (4th ed.). Springer.

Reason:
The source above details how workflow engines record runtime transaction logs, retain state history, and support execution monitoring.
It also explains how workflow instances invoke external web services and APIs, manage runtime exceptions, and handle updates to workflow definitions while instances are actively running.

Task 2: Define the Scenario and the System

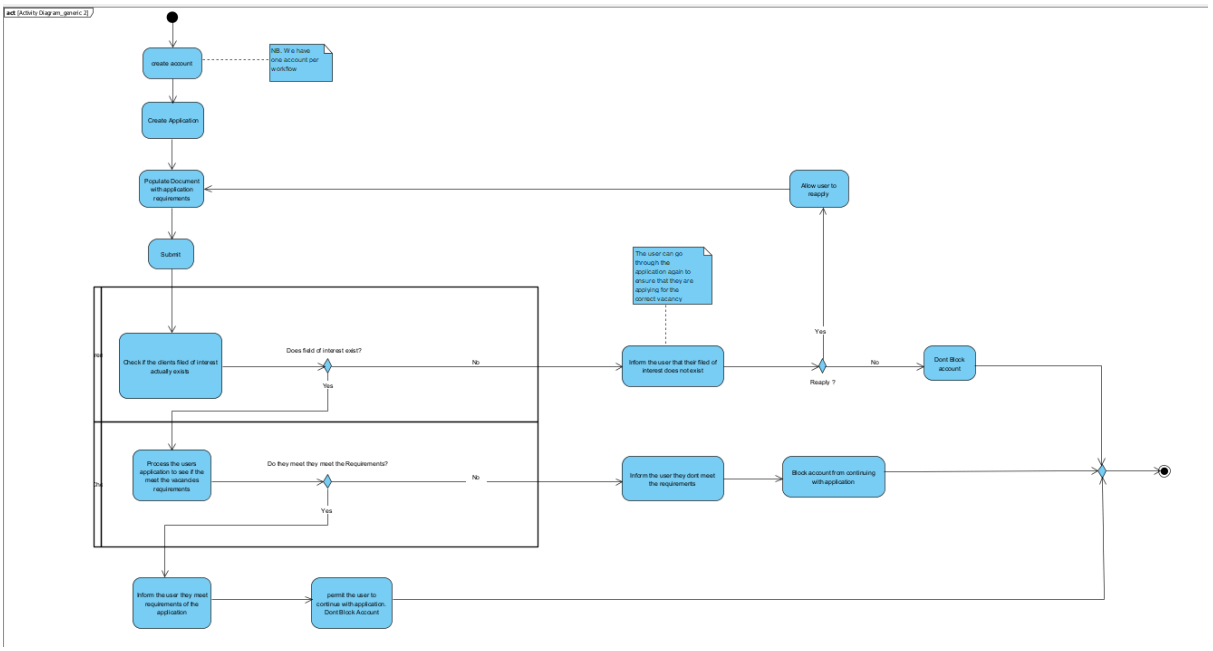
The organisation and environment:
The organisations we will be making this system for will be that of a financial or educational organisation. The system will however work for any other organisation.
The environment is that of an application type system, where a document with data is necessary.
The process being managed:
The process being managed is that of a decision making and validation process. The system takes in data in a certain form, checks it against client set conditions and releases the data as “accepted” or “passed” if it works, otherwise the system fails and clearly states why.
The participants and roles involved:
The participants in this case will be that of the user making use of the system through a dedicated TUI (Terminal User Interface) and that of the organisation overseeing the workflow, like a University or financial institution. The user’s role is to enter data into the system, the organisation's role is to set the conditions of which the data will be evaluated against and in what order.
The major pieces of work:
The major pieces of work is that of the WorkflowNodes and subsystems related to them. There are a small handful of nodes that are fully handled within the ConcreteHandlers of the systems as a whole (the entire system depends on the Chain of Responsibility and WorkflowInstance) and the rest is the ConcreteHandler functioning as the middleman taking the WorkflowInstance and the control flow to the necessary subsystem. Some systems observe changes, others make changes and others create brand new entities like accounts.
Important decisions and routing points:
The important decisions will be based on the scenarios presented. In our case that will be the university application system and a loan application system.
University application system:
The university application system will start off with a document, the document will then be passed into a configured workflow, defined by the organisation itself. This will involve checking whether or not the document is properly filled in. It will then move on to see whether simple conditions such as marks, age or funding is ticked off using our dedicated

comparable objects, also set by the university. The system will then send outside requests to external services checking things like the student's criminal record to see if any violent crimes were committed. It will then proceed to manual approval steps, where the dedicated university administrators will have to sign off on a student's application, in our system we naturally couldn't send this to an administrator and as such the very same TUI will present those options. Finally if all that passes the student will be granted an account and the system will print out whether or not the entire process/workflow was successful.

Approvals and Dependencies:
For the application to be completely successful, it depends on all the system's internal conditions, such as academics, documents, criminal record etc. The final approvals lie with the head of the department, faculty and university.
In the case of a loan application, basic age and gross income decisions will be made.
Possible failures and Exceptions:
If the applicant doesn't send in all necessary data for the system to check, exceptions would be applicants that haven't fully completed schooling yet, have international schooling (thus different ways of calculating marks or different formats).
They could also have a criminal record, or simply fail a basic funding check.
This is the same for both loan application and student application.
External Systems:
Some external systems include that of a police service, educational record and so on.
Expected outcome of workflow:
The expected outcome is either approval of the study application, or rejection both of which is sent back to the student and can be reviewed.
In the case of the loan application it will also just set out a rejection in the loan application fails.

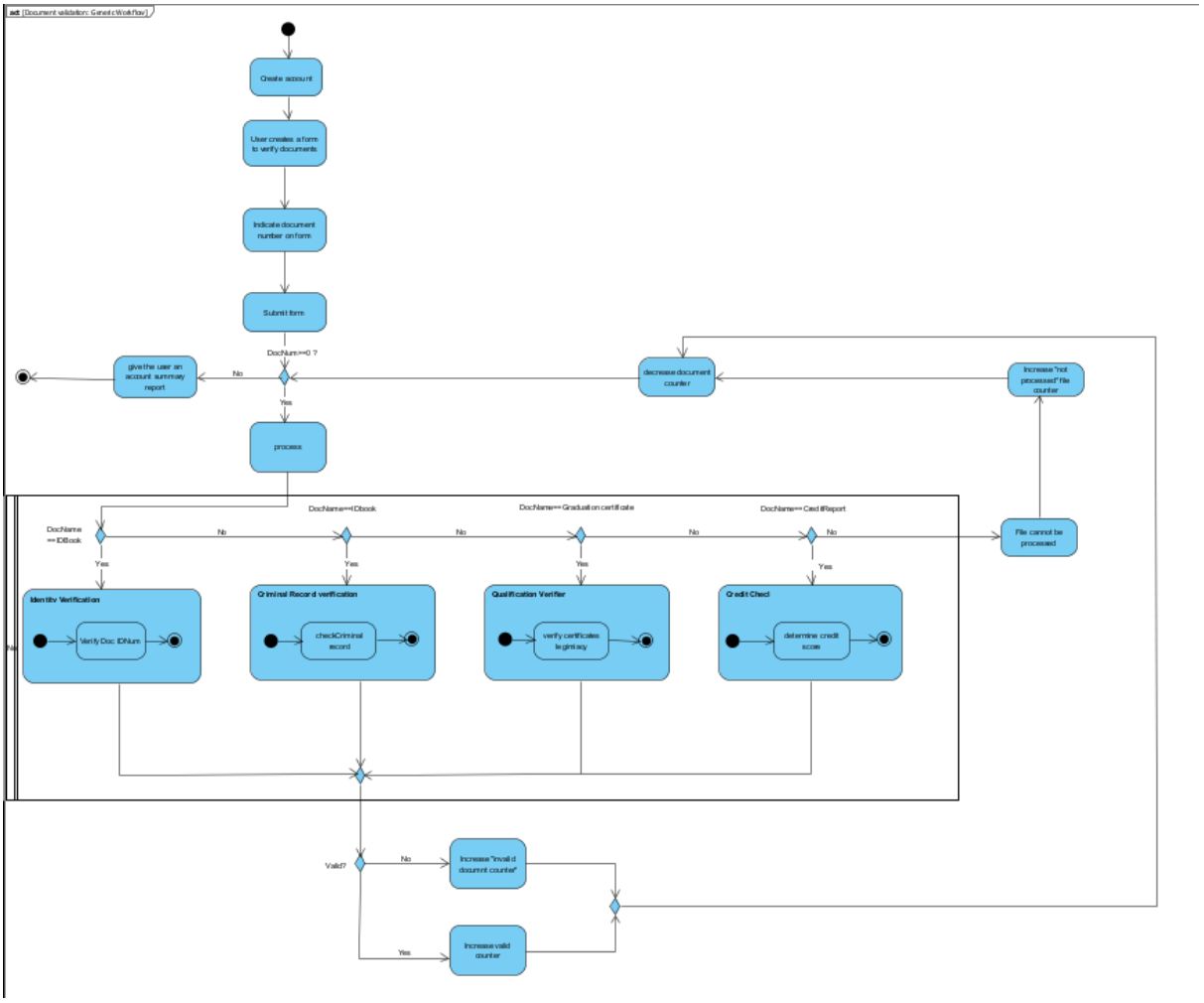
Task 3: UML Activity Diagrams

Task 3.1: generic workflow from creation to final outcome

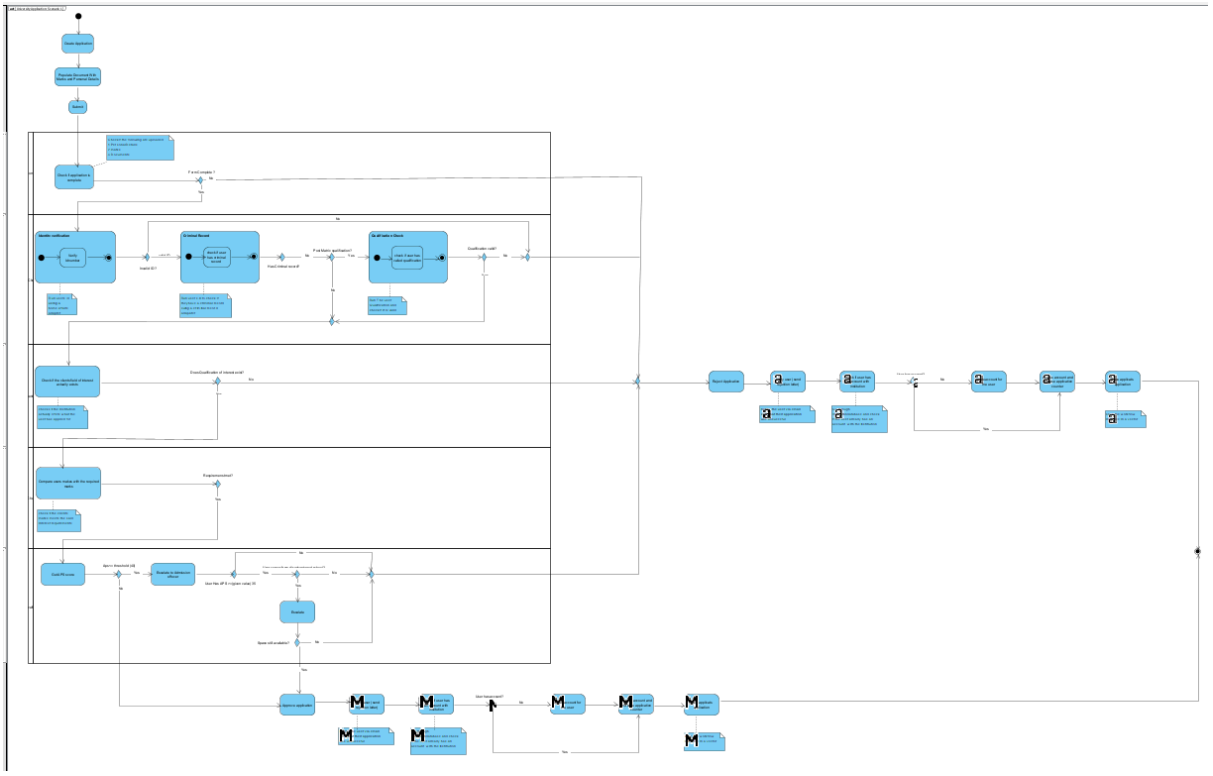


+++

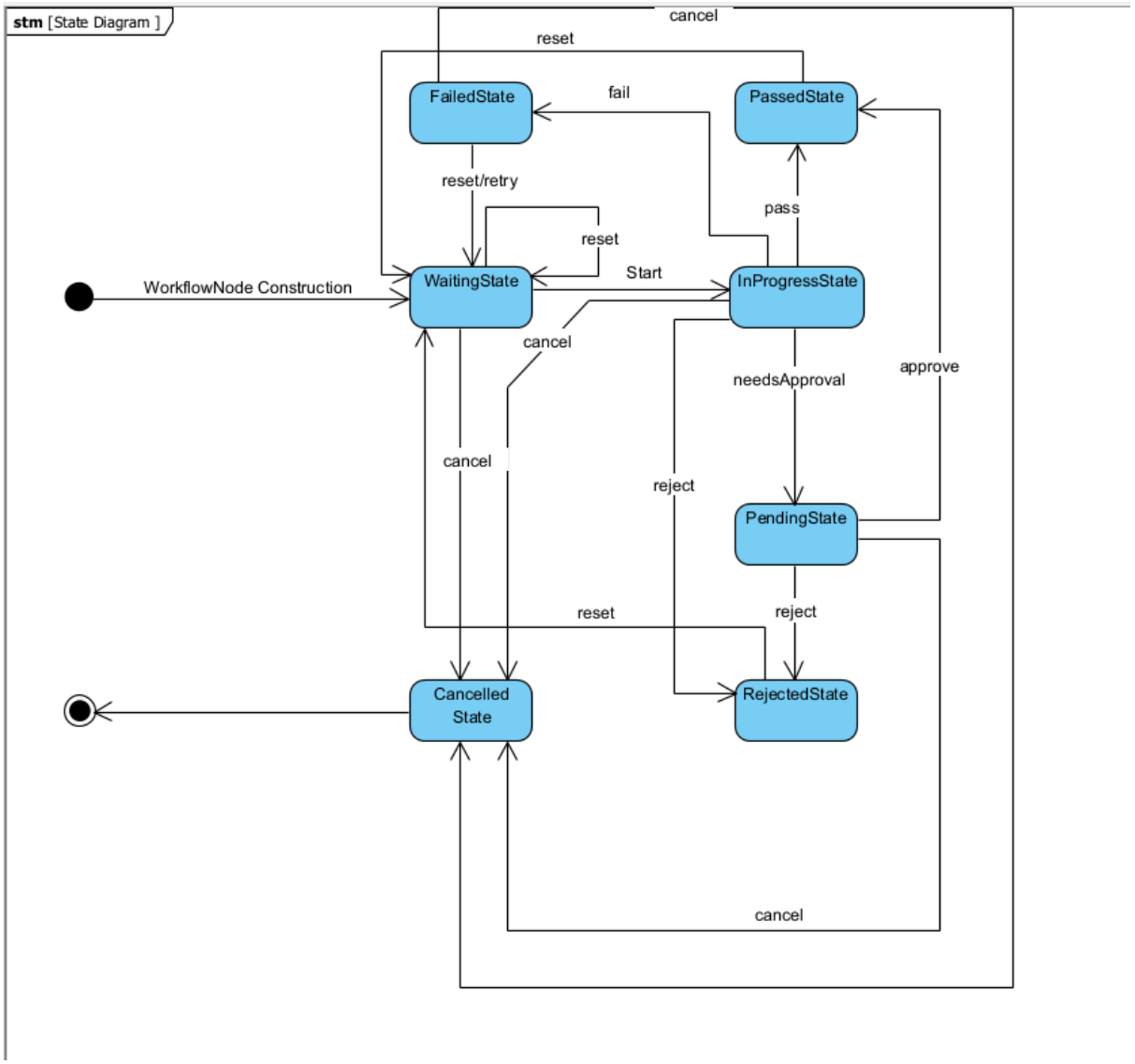
Task 3.2: generic workflow of work item's whole lifecycle



Task 3.3: University scenario workflow



STATE DIAGRAM



Task 4: Design Patterns

Facade:

Design problem in TIVIDY:
The facade design simplifies the rather complex processes of using the main to create workflows, turning complex for loops and switches into functions that can be called and executed at will, saving the client time and complexity
GoF Participant Roles & Classes:
Facade - Facade
Participant Collaboration:
The main/client uses the facade to interact with the TUI and create a user interface.
Varying Elements:

Different parts of the TUI's executing processes are simplified by abstracting it away from the user.

Pattern Justification:

Decouples external user interfaces from internal workflow complexity, preventing changes in core engine mechanics for breaking client-facing applications.

Builder:

Design problem in TIVIDY:

We needed a way to construct a document and dynamically populate the TUI based on the preferences of the client and through the main.

GoF Participant Roles & Classes:

Director - Director
Builder - DocumentBuilder
ConcreteBuilder - DocumentBuilder
Product - Document

Participant Collaboration:

The Director is invoked by the client and makes use of the TUI to not only create the Document product, but also prepare a user interface on to which the fields that should be filled in the document can be filled.

Varying Elements:

The different ways it allows the vector to be converted into a map and how it populates the TUI.

Pattern Justification:

Separates complex workflow construction logic from its structural representation, allowing the same construction process to create diverse workflow instances.

Chain of Responsibility 1:

Design problem in TIVIDY:

How to control the flow of the program and make it dynamic while handling each uniquely.

GoF Participant Roles and Classes:

Handler - WorkflowNode
ConcreteHandlers - ApprovalNode, ExternalCheckNode, RangeCheckNode, DocCompletionCheckNode

Vary Elements:

Naturally each node does something different, interacting with other subsystems and so on.

Pattern Justification:

The ability to step between each node and be able to process each part differently and scaling dynamically is incredibly useful for a generic system, thus the Chain of Responsibility has been used.

Chain of Responsibility Pattern 2:

Design problem in TIVIDY:
Some workflows require human input at some point, for example a Univeristy application needs approval from an application manager, head of faculty and head of department.

GoF Participant Roles & Classes:
Handler - Approver
ConcreteHandler - RuleBasedApprover, ThresholdApprover

Participant Collaboration:
ApprovalNode is a ConcreteHandler of ChainOfResponsibility 2 and is used to link the approver via ApprovalChain.
Varying Elements:
The length of the approval chain, the specific handlers included, and individual escalation thresholds assigned to the handlers based on active workflow strategies. ApprovalPolicy of Approvers also vary.

Pattern Justification:
This pattern allows us to implement Human Approval and also allows us to create a hierarchical chain of human approvers.

Adapter Pattern:

Design Problem in TIVIDY:
Generic workflow steps must use external and legacy services (credit bureau, Home Affairs, exam board, fraud engine, police records), each with its own incompatible interface and its own raw data format, usually with far more data than the workflow needs.

GoF Participant Roles & Classes:
Target - ExternalCheck
Adapter - `CreditCheckAdapter`, `IdentityVerificationAdapter`, `QualificationAdapter`, `FraudCheckAdapter`, `CriminalRecordAdapter`
Adaptee - `LegacyCreditBureau`, `HomeAffairsService`, `ExamBoardAPI`, `FraudScreeningEngine`, `PoliceRecordsDB`
Client - `ExternalCheckNode`

Participant Collaboration:
When the workflow reaches an `ExternalCheckNode`, the node calls `run(document)` on its `ExternalCheck`, without knowing which external system is behind it. The adapter reads only the fields it needs from the `Document` (e.g. the ID number), calls the adaptee's own interface (e.g. `LegacyCreditBureau::fetchScore(id)`), applies its configured rule (e.g. minimum score 620), and converts the raw response into a standard `CheckResult` (passed, readable message, details). `ExternalCheckNode`

then maps the `CheckResult` to PASSED, REJECTED, or FAILED if the external system could not be reached, so the workflow chain continues or stops.

Varying Elements:
The external system's interface and data format, which `Document` fields each check reads, and the decision rule and its thresholds (minimum credit score, required subject marks, maximum fraud risk).

Pattern Justification:
TIVIDY can integrate disparate third-party and legacy systems through one uniform interface, without modifying either the generic engine or the third-party code. Adding a new external check means writing one new adapter; `ExternalCheckNode` and the rest of the engine stay unchanged. Because the field names and thresholds are constructor parameters, the same adapter class serves both industries (e.g. `FraudCheckAdapter` can check a loan amount or a refund amount).

State Pattern:

Design Problem in TIVIDY:
Workflow nodes (such as DocumentCompletenessNode or ApprovalNode) transition through distinct lifecycle stages (Waiting, InProgress, Pending,). The valid operations and behaviors depend entirely on the current state, for example, an approve() action is only valid if the node is Pending, and start() is only valid if it is Waiting. Hardcoding these checks creates complex, tangled conditional logic.

GoF Participant Roles & Classes:
Context -WorkflowNode (maintains an instance of a ConcreteState subclass that defines its current state)
ConcreteState -WaitingState, InProgressState, PendingState, PassedState, RejectedState, FailedState, CancelledState (each implements behavior associated with a specific state
State -NodeState (abstract base class defining the interface for state-specific behavior and default refusal logic)

Participant Collaboration:
When an event occurs (e.g., node->start()), the Context (WorkflowNode) delegates the request to its current State object (NodeState). The active ConcreteState handles the request, performs state-specific logic, and replaces the Context's internal state pointer with a new ConcreteState object (e.g., WaitingState transitions the node by setting its state to InProgressState).

Varying Elements:
The valid actions permitted for each state (e.g., cancel is allowed in Pending, but refused in Passed), the transition rules between states, and the exact behavior upon a refused action.

Pattern Justification:
Eliminates complex, brittle conditional logic checking state flags. By encapsulating state-dependent behavior within dedicated state classes, the system becomes extensible (new states can be added without modifying existing ones) and strictly enforces the workflow rules (e.g., a CancelledState automatically refuses all future events)

Command Pattern:

Design problem in TIVIDY:
Users (staff, applicants) need to act on running workflows (cancel an application, reset a single step, or reset a whole workflow so it can run again). These actions must be checked against the workflow's current state, may be refused (e.g. resetting a cancelled workflow), and every attempt (successful or refused) must be recorded for auditing. Calling the workflow's methods directly from the interface would scatter this checking and logging everywhere.

GoF Participant Roles & Classes:
Command - Command
ConcreteCommand - `CancelWorkflowCommand`, `ResetNodeCommand`, `ResetWorkflowCommand` (a **macro command** made of one `ResetNodeCommand` per step)
Receiver - `WorkflowInstance` (cancel, reset) and `WorkflowNode` (reset of one step)
Invoker - `WorkflowController` (runs every command and keeps the audit log of `CommandLogEntry`'s)
Client - TividyFacade

Participant Collaboration:
When a user cancels or resets a workflow, the client creates the matching command with its receiver and hands it to `WorkflowController::execute()`. The controller records the command's description, calls `execute()`, and logs whether it succeeded. If a state refuses the action, it throws `std::logic_error`; the controller catches it and logs the attempt as *refused* together with the reason, instead of crashing. `ResetWorkflowCommand` first checks that every step can be reset, so a refusal changes nothing (all-or-nothing). It then builds and executes one `ResetNodeCommand` per step, and each step's state decides whether it can return to Waiting. Finally it resets the instance itself.

Varying Elements:
Which action is performed and on which receiver (a whole workflow instance or a single step).

Pattern Justification:
Separates *requesting* an action from *performing* it. The interface never calls the workflow directly, so every user action passes through one place that handles refusals and keeps a complete audit trail. New actions can be added as new

command classes without changing the controller, and the macro command reuses the single-step reset rather than duplicating it.

Observer Pattern:

Design problem in TIVIDY:
When a workflow finishes, other parts of the system need to know about it. For example, an approved applicant needs an account, and the applicant needs to be told the result. We did not want WorkflowInstance to call these classes directly, because then the engine would have to change every time we add something new.

GoF Participant Roles & Classes:
Subject - WorkflowSubject
ConcreteSubject - WorkflowInstance
Observer - WorkflowObserver
ConcreteObserver - AccountCreationObserver, NotificationObserver

Participant Collaboration:
Observers are attached to a WorkflowInstance before it runs. When the workflow is approved, rejected or cancelled, the WorkflowInstance calls **`notify()`**. This calls **`onWorkflowFinished()`** on every attached observer, in the order they were attached. AccountCreationObserver creates an account using the factory, but only if the workflow was approved. NotificationObserver sends the applicant a message with the result.

Varying Elements:
Which observers are attached to a workflow, and what each observer does with the result. For example, an account is only created when the workflow is approved.

Pattern Justification:
The workflow engine does not need to know what happens after a workflow finishes. To add a new action, like logging or sending an email, we just add a new observer without changing WorkflowInstance.

Proxy Pattern:

Design problem in Tividy:
After a workflow an account is created for the client (The party that used the workflow) . The client wants to view their account afterward, but does not want other clients to view their accounts. Staff members need to have access to the account do admin, and auditors need a summary of the account to get information. Different user roles need different access permissions to view accounts. We use protection proxy to solve this problem.

GoF Participant Roles & Classes:
Subject: AccountView
RealSubject: RealAccount
Proxy: AccountProxy

Participant Collaboration:
The TividyFacade creates the AccountProxy and views the account. RealAccount owns the an Account Product from Factory Method and AccountProxy uses AccountRegistry from Factory Method to log client attempts to access accounts.

Varying Elements:
There are different types of UserRoles with different types of AccessLevels: Staff has AccessLevel FullWithNotes for all accounts. Auditor has AccessLevel Summary for all accounts and Client has AccessLevel Full for their account. A Client that tires to access an account different from theirs has AccessLevel Denied.

Pattern Justification:
Accounts can be created for clients and access control is achieved by allowing different levels of access to different types of users. The real account is only revealed to users with the necessary access level.

FACTORY PATTERN:

Design problem in Tividy:
In Tividy different workflow types may need to create different types of accounts to be created. For example the system may need to create a studentAccount or a LoanAccount depending on the type of workflow being processed.
Creating these objects directly inside the workflow management logic would cause the workflow system to be coupled to the concrete account classes.
The Factory pattern addresses this problem by centralising account creation, Instead of allowing the workflow-management components instantiate the concrete account themselves they delegate the creation process to the appropriate factory.

Classes and their GoF participant roles

Creator => AccountFactory
Concrete Creator => FinanceAccountFactory
Concrete Creator => GeneralAccountFactory

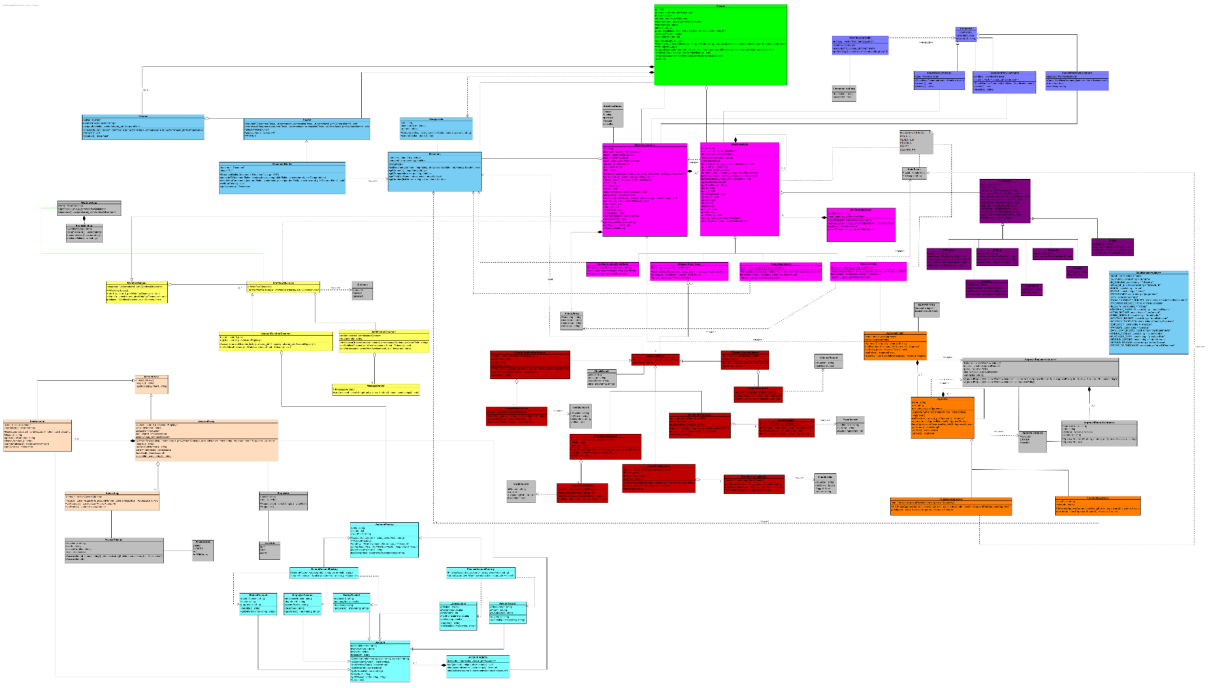
Product => Account
Concrete Products {StudentAccount, EmployeeAccount, LoanAccount}
There is also a AccountRegistry class that maintains and provides access to the created accounts.

Collaboration between participants
The workflow delegates account creation to the appropriate factory. The factory creates and returns the required account through the common Account interface.The Account is then registered in AccountRegistry.

What varies:
The type of account created varies depending on the workflow type

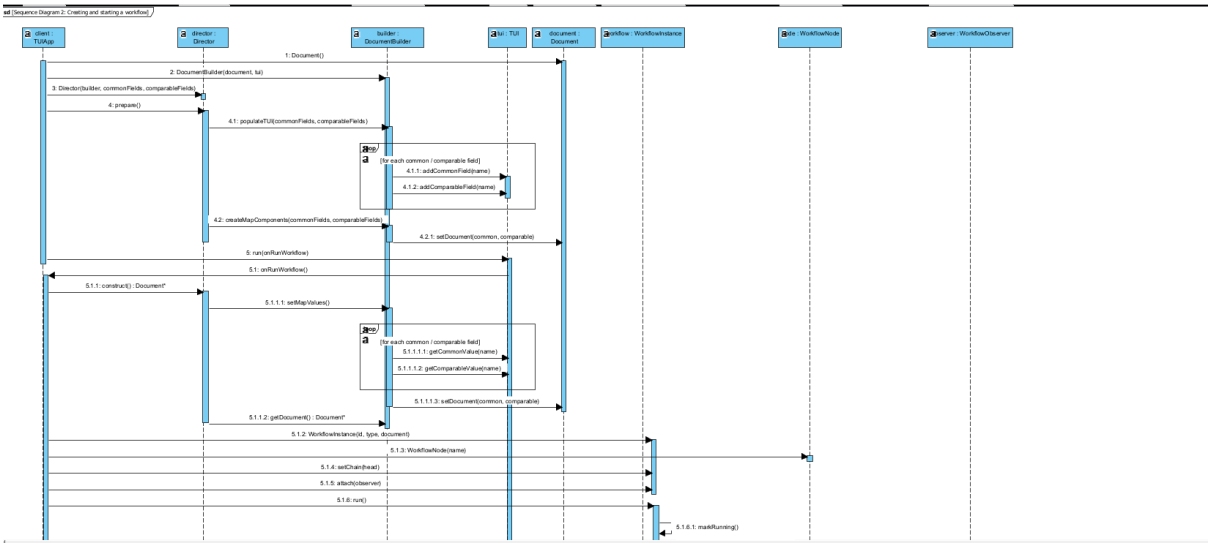
Why is Factory appropriate: It separates account creation from workflow processing, reduces coupling and makes it easier to introduce new account types.

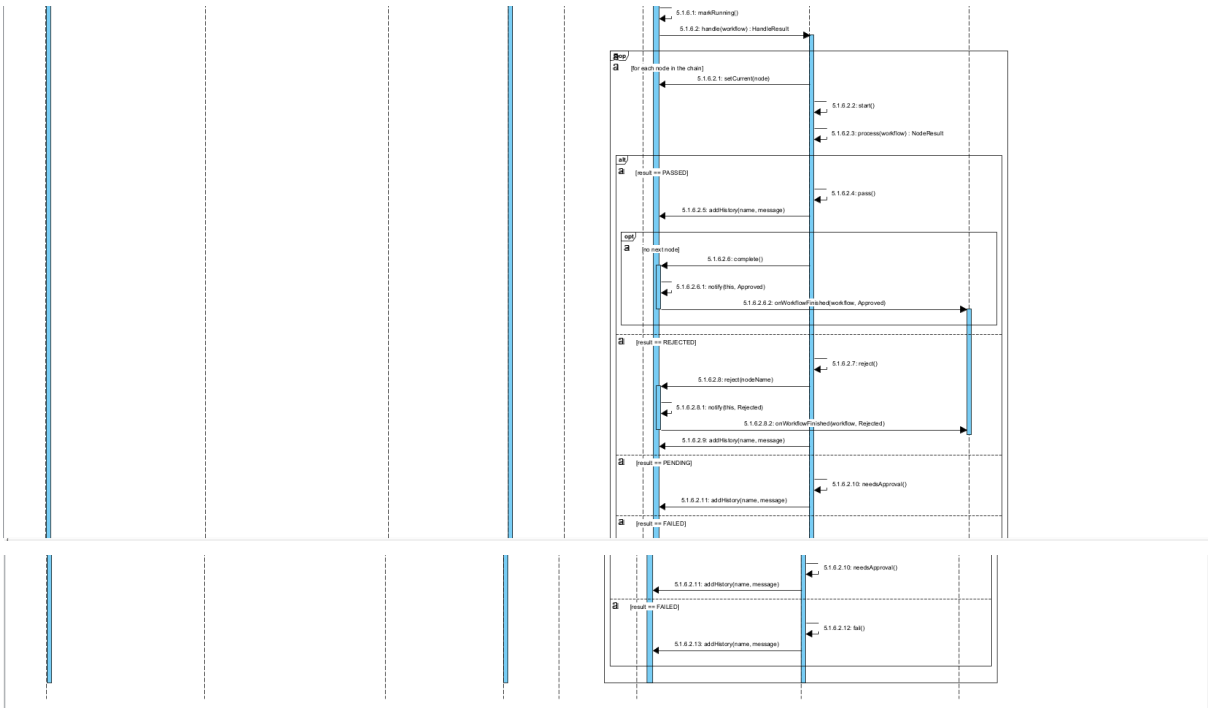
Task 5: UML Class Diagram



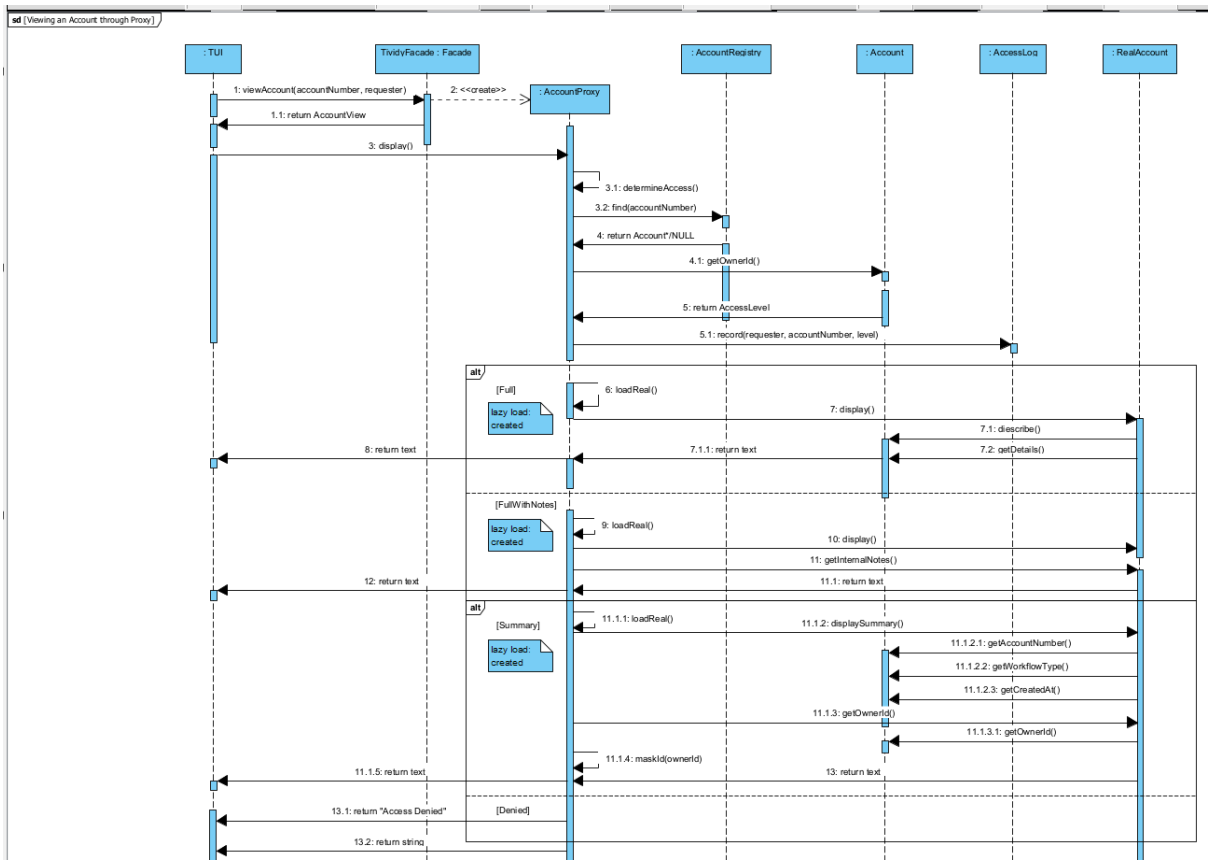
Task 6: Runtime Behaviour Diagrams

Sequence Diagram1 Creating and starting a University application workflow:





Sequence diagram 2: Account Viewing via proxy



Task 7: Design Decisions and Revisions

Week 1:
Our first week we were discussing how to approach the problem of the design having to be “generic” this lead us to the decision to interpret the word “generic” within the context of a specific industry, or specific workflow requirements, with the acknowledgement that creating a workflow that perfectly suits any business, institution or entity’s need is close to impossible.

The other problem we faced is over-spiraling complexity, how to keep the system modular, and maintainable and how to stop the inadvertent “boxing in” of the workflow engine. This leads us to another interpretation of consider that the physical code has to be easily changeable or dynamic, assuming that whatever institution who uses this system will also want to add things like States, Strategies, and underlying conditions that can’t be done through a simple interface itself. We will adjust accordingly in case the tutor points out that the system has to be adjustable through the client side more than it has to be adjustable within the codebase.

Week 2:
Week 2 we spent much time wrestling with how the generic system is supposed to work, first interpreting it as generic within the context of education. This was later refuted and we decided to switch the system into that of a generic type system for multiple system, but that uses the exact same workflow and condition type checks. This was later also determined to be false. This forced a complete rework and the birth of the node based workflow engine.

Week 3:
We spent the final week of the project refining and working through multiple drafts to no avail, eventually we settled on a skeletal design and core ideas and decided to just start throwing things together, leaving major integration issues for later. Just so that progress could be made. This allowed us to start having the shape of our system by the end of the week, but only a functional system by the day of submission.
Our final system has 4 distinct nodes and 2 distinct processes.
The nodes are the parts that are primarily shuffled around by clients to use and the process tends to be at the very end, they can however be emitted if the client so chooses.
The entire system allows for changes to happen within the main, but it hardly relies on the document to be processed immediately.

Task 8: Github workflow

https://github.com/MandlaKhoza/COS214_Practical_6

Our GitHub workflow consisted of using different branches of different subsystems all following a rough naming scheme of feature/subsystem_[team member/s]. This allowed us to keep track of who is doing what and when.
Finally the integration happened bit by bit, first building the base using Builder+COR1 and then finally using that base to test every other subsystem. We finally then proceeded to create an integration branch where we tested the early stages of integration while sorting out

structural issues. We then moved on to an integration branch with the TUI to keep the functional system separate from the graphical interface.
Overall this system was incredibly successful and taught us much about how integration of these types of systems work.

Extra Information: Early discussion and ideas

Explanation:

1. Applicant submit applicationForm, TIVIDY creates:
    - An application object containing the submitted information.
    - It then starts a WorkFlowInstance for that application
    - The instance follows the uni application WorkFlowDefinition

2. Completeness check:
    - Check that all the required information is provided. If anything is missing (Print a summary of what happened and whats missing - memento can provide history) we abort the application.

3. Degree Verification:
    - Check whether the application meets the minimum requirements.
    - Chain of responsibility

4. Update status:
    - Update the application's status as it progresses
5. Human Approval
    - An authorised person reviews the application and approves or reject it.
6. Account Creation:
    - If approved, automatically create an account for the user
7. Notification:
    Notify the applicant of the outcome

Application Form:

: ApplicationForm {
fullName : string
surname : string
idNumber : string
gender : boolean
emailAddress : string
phoneNumber : string
residentialAddress : string
degree: Degree*
}

: Degree {
name : string
nqfLevel : int

institution : string
averageMark : int
}

Workflow Idea:
Create application object
Step 0: Application Object Creation
Step 1: Check if everything is filled in
Step 2: Check degree
Step 3: update state
Step 4: Check Criminal record
Step 5: Human approval
Step 6: Create user account
Step 7: Assign user to applied place/institution

Another workflow idea:
Phase 1. Payload intake and Structure assembly (Builder + Facade)
Phase 2: Automated Verification and Eligibility Safeguards (Template Method + Adapter + State)
Phase 3: Dynamic branching and Evaluation (Strategy + Composite)
Phase 4: Multi-level Approval and Escalation (Chain of Responsibility)
Phase A: Audit Snapshot and Decision Dispatch (Memento + Observer)

Template method object:
Preconditions
execution/conditions
Postconditions
transitionState

Memento:
Summary of steps, stores each decision made here

Builder:
Initial object, the basis of the application

Facade:
Interface for the application/ Builder as well

Adapter:
Criminal Record check

Strategy:
Fast track application, normal application, slow application
Different checks depending on marks, location etc.

Chain of Responsibility:
Multilevel approval: University -> faculty -> department
State:
Phase of each approval (can be used in just in the multilevel approvals, or also in some other approvals in former phases)

Observer:
Monitors each action and uses the memento as “storage”

Composite:
ConditionGroups and Conditions

Another idea:

Step 1: Payload→ get data and documents
Step 2: Define the workflow→ Build the University admission definition: Builder and composite
Step 3: Start→ Applicant submits, each instance is cloned: Prototype
Step 4: Completeness check→ Validate fields and docs. Wrap them in validation labels and loopm back if not complete/ error occurs: decorator, memento
Step 5: Verification→ check a couple of things and fork/join. Check criminal record, matric results, proof of payment. Use adapter for emails to bank and police: Strategy and Adapter.
Step6: Approval. Go through different chains of personnel and approve: Chain of Responsibility
Step 7: Offer→ Inform the student of reject/approval: adapter
Step 8: Registration→ If student is accepted, create student record and stuff like student number: Command and Adapter
Step 9: Reporting and memento: Applicant can cancel registration or appeal rejection, which rolls back to step 6: Memento